

intensive than only serving static files, so if you anticipate a lot of traffic, be prepared for a lot of server load and use caching techniques carefully.

The first thing you should consider before using SSR in your app is if you really need it. It all comes down to how critical time-to-content is for your app. SSR, for example, would be overkill if you were creating an internal dashboard where a few hundred milliseconds on initial load would be insignificant. However, in situations where time-to-content is important, SSR will assist you in achieving the best possible initial load efficiency.

Prerendering vs. SSR

If you're just looking at SSR to boost the SEO of a few marketing pages (e.g., /, /about, /contact, etc.), you're possibly better off with prerendering. Prerendering, as opposed to utilizing a database server to compile HTML on the fly, merely creates static HTML files for individual routes at build time. The benefit is that prerendering is far easier to set up and helps you to maintain your frontend as a mostly static site.

If you're using webpack, the prerender-spa-plugin makes it simple to incorporate prerendering. It has been thoroughly reviewed for Vue apps.

Security

- **Rule number one is to never use untrusted templates.**

When using Vue, the most basic protection guideline is to never use non-trusted information as a component prototype. Allowing arbitrary JavaScript execution in your program is similar to allowing arbitrary JavaScript execution - which, worse, may lead to server violations if the code is executed during server-side rendering. An example of this is:

```
• new Vue({
•   el: '#app',
•   template: '<div>' + userProvidedString + '</div>' //
NEVER DO THIS
• })
```

Vue templates are compiled into JavaScript, and expressions within them are performed throughout the rendering phase. Since the expressions are tested against a particular rendering background, it is impossible for a system like Vue to fully protect you from possible malicious code execution without incurring unreasonable performance overhead due to the difficulty of potential global execution environments. The most simple approach to